

x3d_mcp: A Model Context Protocol Server for AI-Assisted X3D Authoring

Nikhil Narra
Web3D Consortium
USA

Nicholas Polys
Virginia Tech / Web3D Consortium
USA

Rick Lentz
Web3D Consortium
USA

Don Brutzman
Web3D Consortium
USA

Abstract

Large language models (LLMs) are increasingly capable of generating structured technical content, but producing valid Extensible 3D (X3D) scenes is brittle in practice: without grounding in the authoritative specification, LLMs hallucinate node names, misuse field types, and emit broken ROUTE connections that fail XSD validation silently. We present x3d_mcp, an open-source Model Context Protocol (MCP) server that grounds LLM-driven X3D authoring in the ISO/IEC 19775 standard via the X3D Unified Object Model (X3DUOM), the X3D 4.0 and 4.1 XML Schemas, the X3D-Edit tooltip profile, and the type-safe X3DPSAIL *x3d.py* Python library. The server is the result of merging two independent implementations developed in parallel within the Web3D Consortium AI with X3D Working Group, now merged and maintained as an official Web3D Consortium open-source project. It currently exposes 28 tools and four guided prompts across nine modules that cover the full X3D authoring lifecycle (specification discovery, scene synthesis, editing, animation, encoding conversion, validation, and X3DOM rendering in HTML pages); every artifact is built through a four-level validation pipeline: type-safe construction, JSON-Schema validation of tool I/O, XML Schema validation, and authoring-level semantic checks. We demonstrate the server with four end-to-end workflows, including an iteratively refined animated Solar System rendered in X_ITE and a Doha, Qatar city scene assembled from live Google Maps APIs for the Web3D 2026 conference, showing that specification-grounded MCP authoring delivers valid, semantically consistent X3D where unstructured LLM generation does not.

CCS Concepts

• **Computing methodologies** → **Graphics systems and interfaces**; *Artificial intelligence*; • **Information systems** → *World Wide Web*.

Keywords

X3D, X3D 4.1, Model Context Protocol, MCP, LLM, Generative AI, Web3D, Semantic Validation

ACM Reference Format:

Nikhil Narra, Rick Lentz, Nicholas Polys, and Don Brutzman. 2026. x3d_mcp: A Model Context Protocol Server for AI-Assisted X3D Authoring. In *The 31st International Conference on 3D Web Technology (Web3D '26)*, October 13–15, 2026, Doha, Qatar. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3822516.3834882>

1 Introduction

Large language models (LLMs) have become competent generators of code, prose, and structured data, and there is rapidly growing community interest in applying them to 3D content authoring. Extensible 3D (X3D) is an attractive target for this kind of automation: it is the open ISO/IEC 19775 International Standard for declarative 3D on the Web [ISO/IEC 2023], with native XML, JSON, and ClassicVRML encodings and a mature ecosystem of tools, run-time applications, and authoring environments (X_ITE, X3DOM, Castle Game Engine, X3D-Edit). In principle, an LLM should be able to read a natural-language description and emit a complete, valid X3D scene.

In practice, raw LLM output rarely survives schema validation. Across our own experiments at the Virginia Tech Advanced Research Computing (VTARC) Visionarium and in prior community work [Polys et al. 2024], models invent node names that do not exist in the X3D specification, place geometry under unsupported parents, supply field values of the wrong field type (e.g., `SFColor` where `MFColor` is required), wire ROUTE connections between fields whose `accessType` or type signatures do not match, and reuse DEF identifiers that were never declared. The resulting X3D output is plausible XML that initially parses cleanly without error, but it fails XSD validation, fails semantic validation, and renders incorrectly or unpredictably. As the draft X3D 4.1 introduces six new nodes (`EnvironmentLight`, `FontLibrary`, `GaussianSplats`, `HAnimPose`, `InlineGeometry`, `Tangent`) and the X3D Unified Object Model (X3DUOM) continues to evolve in tandem with the X3D Architecture, the gap between training-data familiarity and current-standard correctness only widens.

The underlying problem is not the model's reasoning capacity but rather its lack of grounding. The X3D specification—roughly 265 concrete node types, full inheritance with `additionalInheritance`, per-field type and access-type metadata, container-field rules, and authoring tooltips with hints and warnings—is already encoded in machine-readable form (in the object-oriented X3D XML Schema and the fully elaborated X3DUOM), but a stock LLM has no runtime access to such guiding information. Scene generation collapses to pattern matching



This work is licensed under a Creative Commons Attribution 4.0 International License. *Web3D '26, Doha, Qatar*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2810-5/2026/10

<https://doi.org/10.1145/3822516.3834882>

relative to whatever fragmentary X3D content appeared during pretraining.

The Model Context Protocol (MCP) [Anthropic 2024b], introduced in 2024 and now adopted by major AI providers, supplies exactly the missing piece: an open collection of model-independent conventions for exposing tools, data sources, and structured operations to an LLM at inference time. With MCP, the model can query the authoritative specification, request construction from a type-safe library, and validate every artifact it produces, all within a single conversation.

In this paper, we present `x3d_mcp`: an open-source MCP server that exposes X3D authoring capabilities to LLMs, grounded in the official X3D 4.0 (plus draft 4.1) standards together with the X3DUOM. The server is the result of a convergence of two independent implementations (`ricklentz/x3d_mcp` and `niknarra/x3d-mcp` on GitHub) that developed in parallel for the Web3D Consortium AI with X3D Working Group and have since been merged into a community-maintained repository at `Web3DConsortium/x3d_mcp`. We argue that this independent convergence on substantially the same architecture is itself evidence that the design space is well understood and that the time is right for a Consortium-sanctioned reference implementation.

Contributions. This paper makes the following contributions:

- **An official Web3D MCP server.** We present the first Web3D Consortium-sanctioned server for X3D, organized as nine modules (workflow, granular, query, validation, scene operations, animation, conversion, rendering, and guided prompts) and adopted as an official open-source project of the Web3D Consortium.
- **Specification-grounded generation.** Every node and field operation is backed by the X3DUOM (approximately 265 concrete nodes with full inheritance), the X3D 4.0 and 4.1 XML Schemas, and the X3D-Edit tooltips, whose authoring hints and warnings are overlaid at load time to enrich every specification response.
- **A four-level validation pipeline.** Generated content is checked at four levels: type-safe construction via X3DPSAIL (`x3d.py`), JSON-Schema validation of MCP tool I/O, XML Schema validation, and semantic checks beyond XSD (DEF/USE consistency, ROUTE type and access-type matching, Shape completeness, missing-viewpoint detection).
- **Workflow and granular authoring modes.** High-level tools (`create_geometry`, `compose_scene`) support single-call scene synthesis, while low-level tools (`create`, `modify`, `remove`, `reparent`) support iterative editing. Both produce the same validated XML.
- **Specification-driven animation.** The animation tool selects the correct interpolator by looking up the target field's type in the X3DUOM (`SFRotation` → `OrientationInterpolator`, `SFVec3f` → `PositionInterpolator`, etc.) and emits complete `TimeSensor` + `Interpolator` + `ROUTE` chains, fully validated.
- **End-to-end browser rendering.** Generated scenes are automatically wrapped in X3DOM (or X_ITE HTML5) pages, with the namespace and tag-case normalization required for HTML5 parsing, so authored content is immediately viewable in a browser.

The remainder of the paper is organized as follows. Section 2 reviews X3D, MCP, and prior work on LLM-assisted X3D authoring. Section 3 details the system architecture and the four-level validation pipeline. Section 4 surveys the X3D assets the server is

grounded in. Section 5 walks through four demonstration workflows, including an iteratively refined animated Solar System rendered in X_ITE and a Doha, Qatar city scene built for Web3D 2026 from live Google APIs. Section 6 discusses local and online hosting considerations. Section 7 concludes with limitations and recommendations for future work.

2 Background and Related Work

2.1 Extensible 3D (X3D)

X3D is the open, royalty-free International Standard (ISO/IEC 19775) for declarative, interactive 3D content on the Web [ISO/IEC 2023]. A scene is encoded as a hierarchical *scene graph* whose nodes describe geometry, appearance, transforms, sensors, interpolators, and time-based events; node interconnections (events) are expressed as ROUTE elements between typed fields. X3D supports multiple encodings: XML (with a canonical syntax), ClassicVRML, and JSON; it is implemented by browser-based runtimes such as X3DOM [Behr et al. 2009] and X_ITE, the Castle Game Engine for native deployment, and authoring tools including X3D-Edit. The X3D Unified Object Model (X3DUOM) [Brutzman and Web3D Consortium 2024a] is a machine-readable companion to the schema that exposes the full type system—roughly 265 concrete nodes, their inheritance relationships, per-field interface signatures (type, access type, default value, allowed values), along with X3D component and profile membership—in a single XML document. The Python X3D Scene Access Interface Library (X3DPSAIL), distributed as the `x3d.py` package, is auto-generated from the X3DUOM and provides type-safe construction of scene graphs from Python; each is maintained by the Web3D Consortium. A similar X3D Java Scene Access Interface Library (X3DJSAIL) is also available [Brutzman and Web3D Consortium 2020].

2.2 The Model Context Protocol (MCP)

The Model Context Protocol (MCP), introduced by Anthropic in late 2024 and subsequently adopted by major AI providers including OpenAI and Google, is an open standard for connecting large language models to external tools, data sources, and structured operations [Anthropic 2024b]. An MCP server exposes a set of named tools, each with a JSON-Schema input and output specification; an MCP client (typically an LLM-powered desktop application or IDE) discovers these tools at runtime and the language model invokes them by emitting structured tool-call messages. The protocol is network-transport agnostic, but the most common deployment uses stdio communication between a locally launched server process and a desktop client. The Python reference SDK, FastMCP [Anthropic 2024a], allows ordinary Python functions to be registered as MCP tools with decorator-driven schema inference.

The salient property of MCP for specification-grounded authoring is that the model retains its general linguistic capabilities while gaining *precise, authoritative* access to external state. Rather than fine-tuning a model on X3D corpora (which would still leave the model guessing about specification details at inference time and would not adapt to standards revisions), we comprehensively expose the specification itself, together with type-safe constructors, validators, and renderers, and let the model query and compose against them as needed.

2.3 LLM-Assisted X3D Authoring

Recent work has begun to explore LLMs as a generative front end for X3D. Polys et al. [2024] present an early study of LLM-driven X3D scene authoring at Web3D '24, reporting both encouraging qualitative results and recurring failure modes in the generated content across multiple prompt strategies. Their work establishes both the promise and the brittleness of unstructured LLM generation in this domain. It also provided strong motivation for the continuing progress shown in this paper.

We encountered the same brittleness from a different angle. In parallel with our work on *X3Test* [Narra et al. 2025], an automated rendering-validation framework presented at Web3D '25, we experimented with using LLMs to generate varied X3D scenes as test inputs. The generated content was often invalid: scenes failed XSD validation, referenced DEF identifiers that were never declared, or wired ROUTE elements between fields with incompatible accessType, and we frequently found ourselves correcting LLM output by hand before it was usable. That experience pointed towards a more general gap: rather than treat each invalid scene as a downstream test case for *X3Test*, we asked whether the generation step itself might be made grounded by the specification.

Two MCP servers for X3D were developed in parallel in response to this same gap. ricklantz/x3d_mcp emphasized workflow-style tools (high-level operations that synthesize complete scenes from compact JSON descriptions) together with X3DPSAIL-backed construction and integration with the X3D Example Archives. nikanarra/x3d-mcp emphasized granular, specification-lookup-driven tools backed by an in-memory X3DUOM index, tooltip-enriched node descriptions, and a semantic-check layer beyond XSD. The two implementations were developed independently for the Web3D Consortium AI with X3D Working Group and, on close comparison, converged into substantially the same architecture: an MCP server wrapping X3DPSAIL, validation against the bundled X3D schema, and a tool taxonomy organized around the X3D authoring lifecycle. The present paper describes the resulting merger into Web3DConsortium/x3d_mcp, the official Web3D MCP implementation, which preserves the strengths of both predecessors.

3 System Description

3.1 Architecture Overview

The x3d_mcp server is built on the FastMCP Python SDK. Its default network transport is stdio, following the standard MCP launch pattern: the client (e.g., Claude Desktop, Cursor, or any MCP-aware application) spawns the server as a subprocess and exchanges JSON-RPC messages via standard input and standard output. A Streamable HTTP transport is also supported and is currently under active testing, allowing remote clients to attach to a hosted endpoint without spawning a local subprocess; we discuss the trade-offs of each approach in Section 6. The Python X3D Scene Access Interface Library (X3DPSAIL) [Brutzman and Web3D Consortium 2024b], distributed as the x3d.py package, is the primary code path for scene-graph construction: every LLM session that emits X3D content does so by building, serializing, and validating an X3DPSAIL object tree.

At startup, the server loads a bundled set of authoritative X3D assets and keeps them under the server's source tree

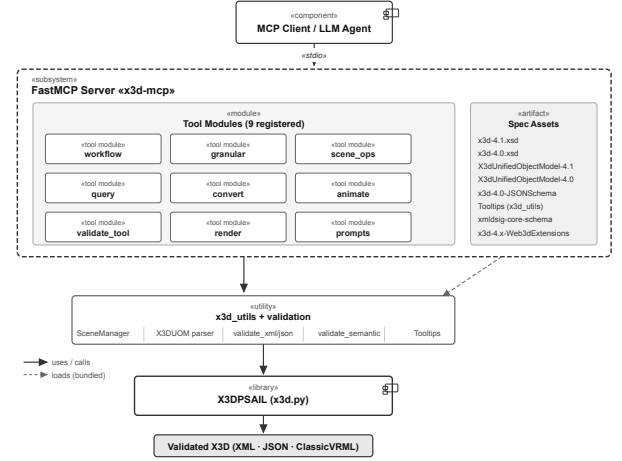


Figure 1: x3d_mcp architecture. The MCP client (e.g., Claude Desktop) invokes tools via JSON-RPC over stdio. The server dispatches each call into one of nine tool modules; scene construction is performed by X3DPSAIL, queries are answered from the in-memory X3DUOM index, and every artifact is validated against the bundled X3D 4.0/4.1 schemas before being returned.

(in `src/validation/schemas/`) so that the system is fully self-contained and requires no network access at runtime. The bundled assets include the X3D 4.0 and 4.1 XML Schemas, the X3D 4.0 and 4.1 Unified Object Models, the X3D-Edit tooltips, the standard X3D DTDs, a JSON-format description of X3D, and the XML digital-signature schema. Tool implementations are organized into nine complementary Python modules under `src/tools/` (workflow, granular, scene_ops, animate, query, validate, convert, render, and prompts), each of which registers its tools with the FastMCP instance at server boot time. One subtlety: X3DPSAIL prints a banner to standard output on first import, which would corrupt the JSON-RPC stream; the server transparently redirects stdout to stderr during tool-module import and restores it afterwards. Figure 1 shows the resulting data flow from client invocation through tool dispatch, X3DPSAIL construction, validation, and serialization.

3.2 Specification Intelligence: X3DUOM and Tooltips

The X3DUOM loader (`src/x3d_utils/x3duom.py`) parses the X3D Unified Object Model XML into an in-memory index keyed by node name. For each concrete node, the loader records the component the node belongs to, its inheritance chain (including multiple interfaces via `additionalInheritance`), the full set of fields with individual type, accessType, default value, and any enumeration constraints, and the containerField rules that determine valid parent-child node relationships. The loader resolves inheritance on demand: when a tool queries a Sphere node, for example, the X3DUOM index has already walked the abstract type chain

(X3DGeometryNode $\rightarrow$ X3DNode) to merge inherited fields automatically. The index is a singleton constructed once at server boot via @lru_cache and reused across all tool calls.

X3DUOM tooltip descriptions are comprehensive, succinct and author-oriented. Authoring users—and increasingly LLM diagnostics—benefit from rich descriptive prose. We overlay the X3D-Edit tooltips (x3d-4.0.profile.xml, originally curated for the X3D-Edit authoring environment by the Web3D Consortium) onto the X3DUOM index at load time. The tooltips file uses XML entity references defined in its DOCTYPE internal subset, which off-the-shelf XML parsers cannot resolve without enabling DTD network loading. The tooltip loader (src/x3d_utils/tooltips.py) instead extracts the entity definitions via regular expression, expands them in-memory, strips the leading [fieldName type default] signature from each tooltip body, and indexes the result by node and field name. The overlay enriches the loader’s node and field descriptions with full prose, authoring hints, and warnings drawn from the X3D-Edit tradition. Such tooltips evolve and are maintained independently by the Web3D Consortium. The enrichment is transparent to tool callers: a single describe_node("Sphere") call returns the X3DUOM type information augmented with the corresponding tooltip text.

3.3 Four-Level Validation Pipeline

The server applies four progressively broader levels of validation; each level catches errors the previous one cannot.

Level 1: Type-safe construction. All scene-graph construction goes through X3DPSAIL. Each X3D node is a Python class whose constructor enforces field types at instantiation: a call such as Material(diffuseColor="red") (a string where an SFColor vector is required) is rejected before any XML is generated. Because X3DPSAIL is also autogenerated from the X3DUOM, its type signatures exactly track the draft specification as the X3D standard evolves.

Level 2: JSON-Schema validation of tool I/O. Every MCP tool exposes a JSON-Schema description of its input parameters. FastMCP enforces the input schema before the tool function is called, rejecting malformed calls at the protocol boundary. This layer ensures, for example, that an LLM cannot supply a string where an array of floats is required for a geometry size parameter: the JSON-Schema check rejects the call before it can reach generation.

Level 3: XML Schema validation. Once an X3D document is serialized, it is validated against the bundled X3D XSD (more than 26,000 lines for X3D 4.0; the 4.1 schema is also bundled and used when the document declares version="4.1"). Schema validation is performed by xmlschema with lxml-backed parsing. Before validation we strip XML Schema Instance attributes (xmlns:xsi, xsi:noNamespaceSchemaLocation) that the lxml.etree.XMLSchema library does not handle cleanly alongside content references.

Level 4: Semantic validation beyond XSD. XSD validation is purely syntactic and structural, and might still accept some constructs that are not strictly correct. The semantic layer (src/validation/semantic.py) performs six classes of check that XSD cannot:

- duplicate DEF identifiers within a single scene;
- DEF/USE consistency: every USE must resolve to an existing DEF;
- Shape completeness: a Shape requires both an appearance and a geometry child to render correctly;
- empty grouping nodes (Transform, Group, etc. with no children);
- ROUTE validity: both endpoints’ DEFs exist, the named fields exist on their respective node types, accessType is compatible from source to sink, and the field types match exactly;
- absence of a Viewpoint (informational only).

The output is a markdown report grouped by severity (errors, warnings, info), suitable for an LLM to read and act on without further parsing.

3.4 Fault-Injection Ablation

To characterize the incremental contribution of each validation level, we ran a fifteen-fault corpus spanning the failure classes of Section 1 through every level each fault can reach; Table 1 reports which levels catch each fault class. Three results stand out. First, every corpus fault is caught by at least one level. Second, five document-level fault classes pass XSD validation entirely and are caught only by the semantic layer—dangling USE references, Shape nodes missing geometry, empty grouping nodes, and both ROUTE fault classes—so disabling Level 4 admits all five silently. Third, the XSD catches duplicate DEF identifiers (the attribute is typed xs:ID) but not dangling USE references (USE is not typed xs:IDREF), a precise asymmetry that by itself motivates semantic checking beyond the schema. This structural ablation establishes which level is *capable* of catching each fault class; the frequency-weighted complement, measuring how often live LLM generation produces each class under baseline and MCP-grounded conditions, is part of the evaluation protocol outlined in Section 7.3. The fault-injection harness is included in the project repository.

Table 1: Fault-injection ablation of the validation pipeline.
Key: ✓ caught, ○ passes, — not reachable at that level.

Fault class	L1	L2	L3	L4
Invented node type	✓	—	—	—
Wrong field value type	✓	—	—	—
Invented field name	✓	—	—	—
Out-of-range field value	✓	—	—	—
Malformed tool payload	—	✓	—	—
Missing required tool parameter	—	✓	—	—
Unknown element	—	—	✓	✓
Illegal parent for geometry	—	—	✓	✓
Malformed attribute value	—	—	✓	✓
Duplicate DEF	—	—	✓	✓
USE without DEF	—	—	○	✓
Shape missing geometry	—	—	○	✓
Empty grouping node	—	—	○	✓
ROUTE type mismatch	—	—	○	✓
ROUTE to missing DEF	—	—	○	✓

3.5 Tool Taxonomy and Authoring Modes

The nine tool modules are organized around the X3D authoring lifecycle, summarized in Table 2. Across these modules the server currently exposes 28 MCP tools and four guided MCP prompts, with more expected as the project continues to evolve.

The server supports two complementary authoring modes. **Workflow mode** (the workflow module) offers high-level, single-call tools—`create_scene` (natural-language description → complete X3D document), `create_geometry` (a single shape with material, color, and size), and `compose_scene` (a list of object descriptors plus viewpoint, lighting, and background → a full scene)—each of which returns a finished, validated document in one round trip. Workflow tools are appropriate for cases where the user wants a complete scene from a single instruction.

Granular mode (the granular module) offers a stateful session in which the LLM constructs a scene incrementally: `create_node` returns a handle, `set_field` mutates an unrendered node, `def_node` and `use_node` establish reuse, `add_child` and `add_route` compose the model’s scene graph, and `get_scene` serializes the current state in any supported encoding (XML, JSON, or ClassicVRML). The session can be cleared with `reset_scene`. Granular mode suits exploration, iterative refinement, and cases where the LLM benefits from intermediate feedback after each step.

Both modes converge on the same validated output. A workflow-generated scene can subsequently be edited with the stateless `scene_ops` tools (`modify_x3d_node`, `remove_x3d_node`, `move_x3d_node`); a granular session can be exported and validated; both can be animated via the `animate` module, rendered via `render`, or transcoded between XML, ClassicVRML, and JSON encodings via `convert`.

In addition to its tools, the server registers four MCP *prompts*: `build_scene`, `audit_scene`, `animate_scene`, and `convert_to_x3dom_html`. MCP prompts are user-invocable conversational starters that supply task-shaped instructions to the model; here they encode the canonical multi-step workflows so a user can launch an end-to-end scene build with a single command.

Table 2: The nine tool modules of `x3d_mcp`, grouped sequentially by lifecycle phase. The server currently exposes 28 tools and four guided prompts across these modules.

Module	Phase	Representative tools
query	Discovery	<code>list_nodes</code> , <code>describe_node</code> , <code>list_components</code> , <code>list_profiles</code>
workflow	Synthesis	<code>create_scene</code> , <code>create_geometry</code> , <code>compose_scene</code>
granular	Synthesis (stateful)	<code>create_node</code> , <code>set_field</code> , <code>def_node</code> , <code>use_node</code> , <code>add_child</code> , <code>add_route</code> , <code>get_scene</code> , <code>reset_scene</code> , <code>remove_node</code>
scene_ops	Editing	<code>modify_x3d_node</code> , <code>remove_x3d_node</code> , <code>move_x3d_node</code>
animate	Animation	<code>animate_x3d_node</code> , <code>add_x3d_route</code> , <code>x3d_animation_info</code>
validate	Validation	<code>validate_x3d</code> , <code>validate_current_scene</code> , <code>validate_semantic</code>
convert	Encoding	<code>convert_x3d</code> (XML ↔ ClassicVRML ↔ JSON)
render	Output	<code>x3dom_page</code> , <code>x3dom_starter</code>
prompts	Guided	<code>build_scene</code> , <code>audit_scene</code> , <code>animate_scene</code> , <code>convert_to_x3dom_html</code>

4 X3D Assets of Interest

A common, strict, standards-based terminology underlies the entire X3D ecosystem. The XML Schema (XSD), the X3D Unified Object Model (X3DUOM), the X3D-Edit tooltips, the X3DPSAIL `x3d.py`

Python library, and various third-party renderers and authoring tools are all maintained against the same ISO/IEC 19775 X3D Architecture specifications. Changes to the standard propagate through this asset graph in a consistent, semantically aligned way. `x3d_mcp` consumes several of these artifacts directly; the rest are part of the surrounding ecosystem that an `x3d_mcp` user is likely to interact with.

4.1 Authoritative Specification Artifacts

X3D XML Schema (XSD). The XSD defines the canonical syntax for every X3D document: element names, attribute syntax, attribute types (including SF/MF distinctions), required vs. optional content, and per-profile content models. The X3D 4.0 XSD is roughly 26,000 lines, and a corresponding 4.1 schema is maintained for the current standard’s consensus-driven revision undergoing implementation and evaluation review. The XSD is the ground truth for syntactic validity and is consumed by `x3d_mcp` at validation Level 3 (Section 3.3).

X3D Unified Object Model (X3DUOM). The X3DUOM [Brutzman and Web3D Consortium 2024a] is a single XML document that exposes the X3D type system in a form better suited to programmatic introspection than the hard to parse object-oriented XSD: every concrete X3D node is listed with its component, its inheritance chain, its full field list (type, accessType, default value, allowed values, and the containerField rules that govern its valid parent contexts), and all tooltip associations. The X3DUOM is the authoritative reflection layer for the standard, and is the substrate of `x3d_mcp`’s specification-intelligence module (Section 3.2).

X3D-Edit Tooltips. The tooltips file (`x3d-4.0.profile.xml`) is a structured, curated body of human-readable prose that accompanies every individual node and field with descriptions, hints, warnings and reference links. It was originally developed for the X3D-Edit authoring environment and represents decades of Web3D community knowledge about authoring practice. `x3d_mcp` overlays the tooltip profile onto the X3DUOM index at load time so that this prose is available alongside the type information for every query.

X3DPSAIL. The X3DPSAIL Python library (`x3d.py`) [Brutzman and Web3D Consortium 2024b] is *autogenerated* from the X3DUOM. Each concrete node becomes a Python class whose constructor enforces field types and defaults, preserving the SF/MF distinctions of the specification. Because X3DPSAIL is regenerated when the X3DUOM changes, its type signatures also track X3D standards revisions automatically. `x3d_mcp` inherits this property: when the X3D 4.1 standard is finalized and the X3DUOM and X3DPSAIL are updated, the server gains 4.1 node support without any tool-code changes beyond the bundled-asset refresh.

4.2 Renderers and Authoring Tools

X3D content generated by `x3d_mcp` is platform-neutral and can be consumed by any conformant run-time application. Since X3D is a well-established interchange format, it is not possible to list all relevant tools that might import or process such models. We highlight the run-time applications and tools most likely to be paired with the server, many others are included as part of the X3D Resources page.

X3DOM. X3DOM [Behr et al. 2009] integrates X3D into HTML5: an X3D scene is embedded inside an HTML page, and an accompanying JavaScript runtime renders the scene via WebGL. X3DOM is x3d_mcp's default rendering target; the render module emits a complete HTML5 page that loads the X3DOM runtime from CDN and embeds the generated scene with the namespace and tag-case normalization required for HTML5 parsing.

X_ITE. X_ITE [Seelig 2024] is an alternative HTML5/WebGL X3D runtime with broader native support for X3D 4.0/4.1 physically based rendering and for the ClassicVRML and JSON encodings. X_ITE is also becoming a first-class rendering target in x3d_mcp, and generated X3D models are X_ITE-compatible by construction; adding an xite_page tool symmetric to x3dom_html is a natural extension. X_ITE provides numerous import/export and conversion options.

Castle Game Engine and Model Viewer. Castle [Kamburelis 2024] is a cross-platform native engine that consumes X3D as one of its scene formats. Castle is the natural application target for desktop, mobile, and VR deployment of x3d_mcp output without a web wrapper. Castle also provides numerous import/export and conversion options.

X3D-Edit. X3D-Edit is the long-running authoring environment maintained by the Web3D Consortium and is the original source of the tooltip profile. It remains a useful sanity-check workflow: a human author can open and edit MCP-generated content in X3D-Edit to verify correctness in detail, and the tooltip prose displayed by x3d_mcp matches the hints seen in the authoring tool. Such open-source support provides continuity between machine- and human-driven authoring.

X3D Example Archives. The X3D Example Archives, maintained by the Web3D Consortium, offer an extensive open-source corpus of validated X3D scenes spanning the full range of X3D components and profiles. The Archives provide well-tested source for reference scenes and for community contributions; we recommend them as both an authoring exemplar and a likely future fine-tuning corpus for LLM systems whose generative quality complements MCP-grounded execution.

5 Examples

We demonstrate x3d_mcp with four end-to-end workflows, drawn from the project's open example repository¹. Each example reports the natural-language prompt that initiated it, the MCP tool calls made by the language model, and the validation that confirmed the result; the first two examples additionally show a fragment of the generated X3D. The four workflows progress from a single-tool invocation, to a multi-object scene synthesis, to an iteratively refined animated scene rendered in X_ITE, and finally to a real-world integration of x3d_mcp with live external Web APIs, also rendered in X_ITE.

5.1 Single-Call Geometry: A Red Sphere

The simplest workflow uses the create_geometry tool from the workflow module to synthesize a complete X3D document from a single high-level call.

Prompt. "Make me a red sphere."

Tool call. The model emits a single create_geometry call with three parameters:

Listing 1: create_geometry call for the red sphere example.

```
{
  "shape": "sphere",
  "color": [0.9, 0.1, 0.1],
  "size": [1.5]
}
```

Server response. The server constructs an x3d.Sphere of radius 1.5 wrapped in a Shape/Appearance/Material chain, serializes the result, and validates against the bundled X3D 4.0 XSD before returning. The complete response document is:

Listing 2: Generated X3D 4.0 document.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE X3D PUBLIC "ISO//Web3D//DTD X3D 4.0//EN"
  "https://www.web3d.org/specifications/x3d-4.0.dtd">
<X3D profile='Interchange' version='4.0'>
  <Scene>
    <Shape>
      <Appearance>
        <Material diffuseColor='0.9 0.1 0.1' />
      </Appearance>
      <Sphere radius='1.5' />
    </Shape>
  </Scene>
</X3D>
```

Validation. The model follows the generation step with an explicit validate_x3d call, which returns {"valid": true, "errors": []}. Across the four validation levels (Section 3.3), the call exercised: Level 1 (X3DPSAIL constructor type-checked the diffuseColor and radius fields), Level 2 (JSON-Schema validated the model's tool-call payload at the MCP boundary), and Level 3 (XSD validated the document against x3d-4.0.xsd). A bare LLM emitting the same response without MCP grounding would routinely include an invalid DOCTYPE URI, omit the required profile attribute, or place Sphere as a sibling of Shape rather than a child: all common failure modes observed in our earlier experiments (Section 2).

5.2 Composed Multi-Object Scene: A Table with Objects

The compose_scene tool synthesizes a multi-object scene (geometry, lighting, viewpoint, background) in a single call, producing a fully validated document with stable DEF names for downstream editing.

Prompt. "Create a scene with a wooden table, a red apple, and a blue mug on top. Add a viewpoint and warm lighting."

Tool call. The model emits a structured object list, a viewpoint descriptor, a light, and a background color. The condensed payload is:

¹https://github.com/Web3DConsortium/x3d_mcp/tree/main/examples

Listing 3: compose_scene call for the table scene (abridged).

```
{
  "objects": [
    {"shape": "box",      "size": [2, 0.15, 1.5],
     "color": [0.55, 0.27, 0.07], "def_name": "TableTop"},
    {"shape": "cylinder", "size": [0.06, 1],
     "color": [0.45, 0.22, 0.05],
     "translation": [0.7, -0.57, 0.5], "def_name": "Leg1"},
    // Leg2..Leg4 omitted for brevity
    {"shape": "sphere",   "size": [0.25],
     "color": [0.85, 0.1, 0.1],
     "translation": [0.4, 0.35, 0.2], "def_name": "Apple"},
    {"shape": "cylinder", "size": [0.12, 0.35],
     "color": [0.15, 0.4, 0.85],
     "translation": [-0.35, 0.25, -0.1], "def_name": "Mug"}
  ],
  "viewpoint": {"position": [3, 2, 4],
                "orientation": [1, 0, 0, -0.35],
                "description": "Table overview"},
  "lights": [{"type": "directional",
              "direction": [-0.5, -0.8, -0.3],
              "color": [1, 0.95, 0.9], "intensity": 0.8}],
  "background": {"skyColor": [0.9, 0.9, 0.95]}
}
```

Server response. The server builds a single X3D document containing the Background, Viewpoint, DirectionalLight, and seven Transform/Shape pairs, one per object. Each Transform carries the requested translation and the corresponding def_name as its X3D DEF attribute, so the same scene can be subsequently modified with `modify_x3d_node` or animated with `animate_x3d_node` via the DEF handles. Validation reports the document as XSD-valid; the semantic checker reports no errors, confirming that all DEFs are unique, no Shape is missing a child, and there are no orphaned ROUTEs.

What this demonstrates. The composition is a single round trip rather than the seven-to-eight tool calls a granular-mode construction would require. The named DEFs are the bridge from synthesis to editing: subsequent “make the apple bigger” or “animate the mug to rotate” requests can target Apple and Mug directly without re-parsing the scene graph.

5.3 Iterative Refinement and X_ITE Rendering: Animated Solar System

Our third example shows iterative authoring across three conversational turns, with the scene remaining specification-valid throughout (Figure 2).

Turn 1 — “Give me a solar system in X3D.” The model called `compose_scene` once with the Sun and eight planets as the object list, each rendered as a colored Sphere with a per-object translation and a stable def_name (Sun, Mercury, ..., Neptune). The returned document was XSD- and semantically valid.

Turn 2 — “Add an animation.” The model issued one `animate_x3d_node` call per planet. For each, the animation tool

looked up the target SFRotation field in the X3DUOM, selected an OrientationInterpolator, and emitted the TimeSensor, interpolator chain, and two ROUTEs required to drive it. Cycle intervals were varied to suggest relative orbital periods rather than literal time scale.

Turn 3 — “Add textures and validate the scene.” The model used `modify_x3d_node` to attach ImageTexture children to each planet’s Appearance via the Turn 1 DEFs, then called `validate_x3d` and `validate_semantic`; both passed. No tool re-parsed the scene from scratch: the named DEFs remained the stable handles for essential nodes throughout.

What this demonstrates. A workflow tool scaffolds the scene; animation tooling layers specification-driven dynamics without the LLM having to remember the TimeSensor/Interpolator/ROUTE idiom; and a stateless edit pass adds materials by re-targeting the earlier DEFs. Validation at every turn keeps iterative refinement from silently drifting, and X_ITE confirms the same X3D model artifact runs unmodified on X3DOM.

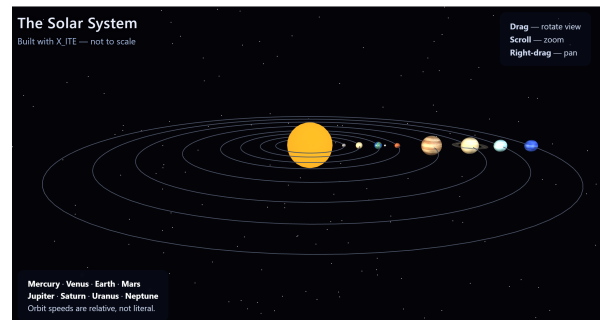


Figure 2: The Solar System scene rendered in X_ITE after three iterative turns: `compose_scene` for geometry and placement, per-planet `animate_x3d_node` calls for TimeSensor/OrientationInterpolator/ROUTE chains, and `modify_x3d_node` for texturing. XSD- and semantically valid at every turn.

5.4 Real-World Scene from Live Web APIs: Doha, Qatar

Our fourth example demonstrates that the integration of `x3d_mcp` with disparate Web-standard data sources becomes substantial when orchestrated by an LLM. It is also the scene we present for the Web3D 2026 conference in Doha, Qatar: a 3D model of the host city assembled from live Google Maps APIs.

Prompt. “Create a 3D scene of Doha, Qatar using real data from Google APIs.”

Orchestration. The model interleaves external Web API calls with `x3d_mcp` tool calls. It first queries the Google Elevation API over a regular grid centered on Doha to retrieve ground-truth elevation samples, and the Google Places API for points of interest across six categories (restaurants, hotels, museums, tourist attractions, shopping malls, and parks), yielding 4,361 places with names, locations, types, and aggregate ratings where available. The model then composes a single X3D scene combining: an ElevationGrid

initialized with the elevation samples and textured with Google Maps satellite imagery; a color-coded Sphere marker per place, with size scaled by rating and a TouchSensor attached for click-to-inspect interaction; a semi-transparent water surface for Doha Bay; a HUD overlay with the Web3D 2026 conference dates and venue; and aerial and ground-level Viewpoint nodes together with directional sun and fill lighting.

Coordinate transformation. The model performs an equirectangular projection from geodetic to scene coordinates at runtime:

$$x = \frac{(\lambda - \lambda_0) \cdot 111000 \cdot \cos \phi_0}{100}, z = -\frac{(\phi - \phi_0) \cdot 111000}{100}, y = \frac{h}{100},$$

where (ϕ, λ) are latitude and longitude, (ϕ_0, λ_0) is the scene center, and h is elevation in meters.

Validation and rendering. The model finalizes by calling `validate_x3d` and `validate_semantic` on the composed scene; both return without errors, confirming that all marker DEFs are unique, every TouchSensor routes cleanly into its target, and every Shape carries the required appearance and geometry pair. Figure 3 shows the scene rendered in X_ITE.

What this demonstrates. The Doha workflow demonstrates two properties that we believe distinguish MCP-grounded authoring from prior LLM-as-3D-generator work. First, an *LLM-orchestrated* pipeline can combine validated 3D synthesis with live data integration: the model is free to call any other MCP server or Web API alongside `x3d_mcp`, and the resulting scene remains X3D-valid because `x3d_mcp` validates every fragment it produces. Second, a natural extension that consumes the Google 3D Tiles photorealistic mesh assets via X3D’s Inline or ExternalProtoDeclare mechanisms would let `x3d_mcp` compose hybrid scenes blending procedurally constructed geometry with photogrammetric content, all validated end-to-end.

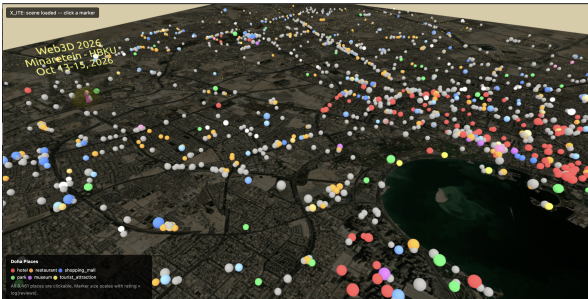


Figure 3: The Doha, Qatar scene rendered in X_ITE. Terrain comes from Google Elevation API, place markers (restaurants, hotels, museums, tourist attractions, shopping malls, parks) from Google Places API, and each rating-colored marker is a selectable anchor linking to its website. The X3D scene graph is fully validated against the X3D 4.0 XSD.

6 Hosting an MCP Server

The typical deployment strategy uses a localhost execution model. Running the MCP server locally (e.g., via Claude Desktop or Cursor) ensures the LLM has immediate, secure access to the user’s

filesystem and tools without exposing an open endpoint to the internet. While online and cloud deployments are feasible, the localhost approach currently provides the lowest friction and highest security for individual developers iterating on 3D scenes. As an early exploration of the cloud-hosted path, we are also exercising the Streamable HTTP transport behind a publicly reachable endpoint provisioned through a containerized deployment; an MCP-aware client can attach by adding the endpoint URL to its configuration file, with no local server install. Best practices continue to evolve for hosting an MCP server, and the GitHub repository will track recommended usage.

7 Conclusions, Limitations, and Future Work

7.1 Summary

We have presented `x3d_mcp`, the first Model Context Protocol server for Extensible 3D, formed by merging two independent implementations into an official Web3D Consortium open-source project. The server grounds LLM-driven X3D authoring by applying the X3D Unified Object Model, the X3D 4.0 and 4.1 XML Schemas, the X3D-Edit tooltip profile, and the type-safe X3DPSAIL `x3d.py` Python library, currently exposing 28 tools and four guided prompts across nine modules that cover the full authoring lifecycle from discovery through validation, scene synthesis, editing, animation, encoding conversion, and rendering within an HTML page. Every artifact the server returns is built through a four-level validation pipeline (X3DPSAIL constructor type-checking, JSON-Schema validation of tool I/O, XSD validation, and authoring-level semantic checks), so a client LLM either receives valid X3D or else receives an actionable error notification. Our demonstrations span a single-call geometry primitive, a multi-object composed scene with named DEFs and lighting, an iteratively refined animated Solar System, and a real-world Doha, Qatar city scene assembled from live Google Maps APIs for the Web3D 2026 conference (the latter two rendered in X_ITE); in each case the server delivered specification-compliant output where prior unstructured LLM generation would not. The work demonstrates the viability of data-driven, server-side, specification-grounded 3D synthesis as a foundation for human-AI collaboration in spatial computing.

7.2 Limitations

We note several honest limitations of the work presented here. First, beyond the structural fault-injection ablation of Section 3.4, the paper reports qualitative demonstrations rather than a controlled, quantitative evaluation of live generation; while we have observed substantial reliability improvements over bare LLM generation in our own daily use, a formal study comparing MCP-grounded and ungrounded LLM authoring on a held-out task suite remains in progress and is not reported here. Second, although the X3D 4.0 and 4.1 XML Schemas and X3DUOMs are both bundled with the server, the X3D 4.1 standard remains in draft status; full end-to-end coverage of the six new nodes (EnvironmentLight, FontLibrary, GaussianSplats, HAnimPose, InlineGeometry, Tangent) will follow ratification of X3D 4.1 and the corresponding X3DPSAIL updates, and remains a target for ongoing implementation and evaluation. Third, the dedicated MCP rendering tool currently emits X3DOM HTML pages only; an analogous X_ITE rendering tool is

in active development. In the meantime, generated content runs in X_ITE unmodified (both the Solar System and Doha examples in this paper were viewed that way), so the practical gap is one of authoring convenience rather than runtime compatibility. Fourth, the server does not yet return multimodal artifacts (rendered screenshots, depth maps, video frames) to the LLM, which would close a useful feedback loop for iterative scene refinement. Fifth, the system has been exercised primarily with Anthropic’s Claude family of models through Claude Desktop; while the MCP protocol is model-agnostic, behavior on other LLM clients has not yet been systematically studied.

7.3 Future Work and Recommendations

We see several productive directions for continuing this work within Web3DConsortium/x3d_mcp, with several implications for the broader Web3D community.

Controlled evaluation. A task suite spanning Tier 1 (single primitive), Tier 2 (DEF/USE, hierarchy, sensors), and Tier 3 (animation, ROUTEs, interaction) scenarios, run under both bare-LLM baseline and MCP-grounded treatment conditions and scored on XSD validity, semantic correctness, and hallucination rate, would provide a quantitative basis for the qualitative claims of this paper.

X3D Schematron quality assurance. The Web3D Consortium maintains an X3D Schematron tool² that encodes a substantial body of authoring heuristics, scene-semantic checks, and best-practice warnings beyond what XSD or our current Level 4 semantic checker captures. Integrating the Schematron ruleset into x3d_mcp’s validation pipeline (either by invoking the existing Schematron processor over generated documents or by porting individual rules into the Python semantic-check module) will extend the specification-grounded validation surface with a substantial body of community-curated quality-assurance rules.

Full X3D 4.1 coverage and additional renderers. Tracking the X3D 4.1 standard through to formal ratification and adding first-class X_ITE and Castle rendering tools will expand the deployment surface and let authors target native applications and immersive run-time tools from the same MCP-driven workflow. Online pages for X3D Resources [Brutzman and Web3D Consortium 2026a] and X3D Scene Authoring Hints [Brutzman and Web3D Consortium 2026b] offer numerous additional possibilities

Multimodal feedback. Returning rendered screenshots and depth or normal-vector information from the renderer back to the LLM would enable visual self-correction (“the apple is floating above the table; lower it by 0.2 m”), which is currently outside the server’s tool surface.

Virtuous-circle progress for X3D ecosystem. Regular review of progress and problems with future generated models will be interesting. We anticipate ongoing improvements to X3D quality assurance (QA) tools, best practices for embedded model metadata, unit-test X3D scenes as these LLM-driven efforts continue to reveal occasional gaps while making useful progress or further adding to x3d_mcp capabilities.

Semantic Web and knowledge graphs. Rules-based queries using the X3D Ontology for Semantic Web [Brutzman and Flotyński 2020] offer interesting possibilities at even-higher levels of abstraction. In particular, exposing the X3D Ontology’s Turtle (.ttl) serialization as an MCP resource, paired with a SPARQL query tool, might let a human modeler (or LLM-based modeling process) pose semantic questions—for example, which node types may legally parent a given geometry node, or which Example Archives scenes exercise a particular component—that neither the XSD nor the X3DUOM index answers directly. Consistent addition of typed metadata to X3D models further deepens such query/reponse possibilities.

We invite the Web3D community and the wider AI-assisted 3D-authoring community to use, extend, and contribute to Web3DConsortium/x3d_mcp. We look forward to deeper collaboration between LLM-driven generation and the open Web3D standards stack.

Acknowledgments

We thank the Web3D Consortium and the broader X3D community for open standards, active collaboration, and insightful discussions.

References

- Anthropic. 2024a. FastMCP: The fast, Pythonic way to build MCP servers. Python SDK for the Model Context Protocol. <https://github.com/modelcontextprotocol/python-sdk>
- Anthropic. 2024b. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>. Specification: <https://modelcontextprotocol.io/specification/>. Accessed 2026.
- Johannes Behr, Peter Eschler, Yvonne Jung, and Michael Zöllner. 2009. X3DOM: a DOM-based HTML5/X3D integration model. In *Proceedings of the 14th International Conference on 3D Web Technology (Web3D '09)*. ACM, New York, NY, USA, 127–135. doi:10.1145/1559764.1559784
- Don Brutzman and Jakub Flotyński. 2020. X3D Ontology for Querying 3D Models on the Semantic Web. 6 pages. doi:10.1145/3424616.3424715
- Don Brutzman and Web3D Consortium. 2020. X3DJJAIL: Java X3D Scene Access Interface Library. Java JAR: x3d; autogenerated from the X3DUOM. <https://www.web3d.org/x3d/stylesheets/python/python.html>
- Don Brutzman and Web3D Consortium. 2024a. X3D Unified Object Model (X3DUOM). Web3D Consortium specification asset. <https://www.web3d.org/specifications/X3DUOM.html> Machine-readable companion to the X3D XML Schema.
- Don Brutzman and Web3D Consortium. 2024b. X3DPSAIL: Python X3D Scene Access Interface Library (x3d.py). PyPI: x3d; autogenerated from the X3DUOM. <https://www.web3d.org/x3d/stylesheets/python/python.html>
- Don Brutzman and Web3D Consortium. 2026a. X3D Resources. web page. <https://www.web3d.org/x3d/content/examples/X3dResources.html>
- Don Brutzman and Web3D Consortium. 2026b. X3D Scene Authoring Hints. web page. <https://www.web3d.org/x3d/content/examples/X3dResources.html>
- ISO/IEC. 2023. *ISO/IEC 19775-1:2023 — Information technology — Computer graphics, image processing and environmental data representation — Extensible 3D (X3D) — Part 1: Architecture and base components*. International Standard. International Organization for Standardization, Geneva, Switzerland. <https://www.web3d.org/specifications/X3Dv4/ISO-IEC19775-1v4.0/Part01/Architecture.html>
- Michalis Kamburelis. 2024. Castle Game Engine: Cross-platform 3D and 2D game engine supporting X3D, glTF, Spine, and many other formats. Open-source game engine. <https://castle-engine.io/>
- Nikhil Narra, Anuj Marisetty, Nicholas Polys, and Ben Sandbrook. 2025. X3Test: A Headless Browser-Based Framework for Automated Performance Benchmarking of X3D/X3DOM Scenes. In *Proceedings of the 30th International Conference on 3D Web Technology (Web3D '25)*. Association for Computing Machinery, New York, NY, USA, 1–5. doi:10.1145/3746237.3746315
- Nicholas Polys, Ayat Mohammed, and Ben Sandbrook. 2024. Prompt Engineering for X3D Object Creation with LLMs. In *Proceedings of the 29th International ACM Conference on 3D Web Technology (Guimarães, Portugal) (Web3D '24)*. Association for Computing Machinery, New York, NY, USA, Article 18, 7 pages. doi:10.1145/3665318.3677159
- Holger Seelig. 2024. X_ITE: A WebGL-based X3D player. Open-source HTML5/WebGL X3D run-time. https://create3000.github.io/x_ite/

²<https://www.web3d.org/x3d/tools/schematron/X3dSchematron.html>